1 Research Plan

Research question: To what extent does the ϵ-cohesive core recover communities in bipartite graphs more completely than existing biclique relaxations, and can it be found efficiently on large graphs?

When creating a new formulation that goes beyond the theoretical and actually proves its utility, there are several things I need to show:

- Does this formulation recover more coherent structure, or larger coherent structure, than other formulations? (Compare it to other methods)? Also, can this structure be used to train embeddings? How much more effective are embeddings trained on this structure than others?
- What is the most efficient way this structure can be found? What are some algorithms that are less efficient but often recover larger structure? Is there a provable brute-force algorithm?

My laptop runs Ubuntu with an Intel(R) Core(TM) Ultra 7 258V CPU, 8 threads, and 32 gigabytes of RAM. As of now, my plan will be to use it for all the brute-force graph tests. If I can secure funding, I will rent a much faster computer for long-running tests.

1.1 What Graphs Am I Going to Use?

For graphs where I need ground-truth communities, I can use:

- Orkut Communities
- Amazon Products Frequently Bought Together
- LiveJournal Connections
- DBLP Collaboration Network

For graphs where I don't need ground-truth communities, I can use the following:

- Open-source bipartite graph repository `http://konect.cc` (600+ graphs with more than 10,000 nodes)
- Amazon review graphs at `https://amazon-reviews-2023.github.io/`
- Yelp Open Data at `https://business.yelp.com/data/resources/open-dataset/`

1.2 Comparing to Other Formulations

There are many different bipartite structures that have been proposed. I need to prove that my structure idea recovers more communities than any of the other methods. Below is a table of every structure I will test on, the hyperparameters that need to be varied, and a list of all the algorithms I could find with their time complexities.

I will run the following tests:

Table 1: Your Table Title

Formulation	Hyperparameters	Algorithms
k-Biplex	k	
Vertex-based (α, β)-biclique	α, β	
Density-based γ-biclique	γ	
k-Defective Biclique	k, θ	
Jaccard-Similarity Bicliques[1]	None	

1.2.1 Planted Edge Recovery

For each formulation, take each real-world dataset. Choose a random group of T right-side nodes and S right-side nodes who are connected to group A. Connect the edges in T and S with prbability p. Connect all other edges with probability q for added background noise.

Run biclique-style methods and the ϵ-CC, and see which one recovers the most nodes in T and S. Report recall and precision for each, varying T, S, p, and q.

I will likely vary $T, S \in \{10, 20, 30, 50\}$, $p \in \{0.5, 0.6, 0.7, 0.8, 0.9, 1\}$, and $q \in \{0, 0.01, 0.02, 0.03, 0.04, 0.05, 0.1, 0.2, 0.3, 0.5\}$.

1.2.2 Held-Out Edge Recovery

For γ-bicliques, k-biplexes, and the ϵ-CC, take each real-world dataset. Remove a random fraction p of the edges, then run each formulation. For each formulation, test the number of held-out edges that that appear in its identified structure, as well as the percentage of edges in the structure that were held out. Report the precision and recall of edges identified from the missing set. This implies how redundant the structure is to noise or missing edges. Vary $p \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$.

Additionally, I will need to show how solution qualtiy varies across selection of different hyperparameters. I will vary θ and ϵ show a curve of how solution size, and similarity to actual community for the ground-truth datasets, change across settings. I will also show the recovery of ground-truth structure for different distance metrics: Jaccard similarity of neighbors, diffusion distance, distance for OpenAI-embeddings, and any other metrics I can think of.

Clauset et al., 2008 provides precedent for holding out a certain number of edges and seeing what percentage the formulation implies.

1.2.3 Ground-Truth Comparison

For graphs with ground-truth communities, I will report the overlap between the largest identified structure and actual communities, as well as how many nodes in a real community a formulation was able to identify.

Yang et al. (2013) gives precedent for evaluating community detection of different algorithms.

1.3 Recommendation

1.3.1 ϵ-Cohesive-Core-Based Recommendation

Let $\text{sim}(u, u')$ be the Jaccard similarity of the cores that u and u' belong to. This way, a higher similarity implies that two users are part of more niche groups together. We can then use this to hold out edges, and then make recommendations for each user based on the product with the highest similarity to the user. If a product they actually bought is in the top 10 recommendations, then this does pretty well at the top-10. Report the MRR.

Specifically, let $\text{sim}(u, u') = \frac{\mathcal{C}(u) \cap \mathcal{C}(u')}{\mathcal{C}(u) \cup \mathcal{C}(u')}$, where $\mathcal{C}(u)$ is the set of ϵ-CCs that u belongs to. If that turns out to be computationally intractable, then instead $\mathcal{C}(u)$ can instead be the set of nodes in the top-K ϵ-CCs that u is a part of.

The score of an item to a user is

$$\text{score}(u, i) = \sum_{u' \in U \text{s.t.} (u', i) \in E_{\text{train}}} \text{sim}(u, u')$$

This has some precedent with PinSage (Ying et al., 2018), which explicitly reported MRR to demonstrate that its recommendation system beat previous models.

1.3.2 Embedding-Based Recommendation

For each real-world graph, hold out a fraction p of the edges, then train an embedding using ϵ-CCs under a distance like Jaccard similarity that doesn't require a learned metric. Then use this to figure out which users are most similar to each other. Add one edge at a time, and then if one user buys a product, suggest it to super similar users and see how often it correctly predicts products that users would want.

Use LightGCN, which already establishes precedent for training a bipartite embedding. Currently, the loss term is $\mathcal{L} = \mathcal{L}_{\text{BPR}} + \lambda \mathcal{L}_2$. I can add an $\mathcal{L}_{\text{core}}$ term that rewards making embeddings for two nodes in the same ϵ-CC point in the same direction, with a parameter α that controls how much this heuristic is weighted.

For each core, the loss for nodes on the items side is defined to penalize distance between two nodes in the same ϵ-CC:

$$\ell_U(C) = \frac{1}{\binom{|U_C|}{2}} \sum_{\substack{u, u' \in S_C \\ u < u'}} (1 - \cos(u, u'))$$

where cos is cosine similarity between LightGCN's embedding of two nodes.

A similar function can be defined for $\ell_V(C)$.

I actually am not sure if this particular training method has precedent, but that's also because my idea of a bipartite structure that doesn't require all-to-all connection is new. This paper uses a similar local-structure cost term to encode biclique structure. It also includes a global structure term that I may want to

look into as I experiment with embeddings. Of course, LightGCN also provides precedent, since it's the model I will be using to train my own embeddinngs.

1.4 Efficiently Finding ϵ-CCs

The best approach to identifying large ϵ-CC depends on the graph size.

For graphs with $|V| \leq 10^2$, a brute-force method is still tractable to identify the best solutions. For sparse graphs with $\leq 3 \times 10^4$ nodes, an algorithm that runs in $\mathcal{O}(|V|^2)$ is tractable enough. For dense graphs, the threshold drops to $|V| \leq 5 \times 10^2$. Past these limits, any heuristic must run in $\mathcal{O}(|V|log|V| + |E|log|E|)$ or less, and preferably without the $log|E|$ term.

Because of all these different needs, I will propose several different algorithms with varying complexities and time them on graphs up to the largest tractable size I can. I will examine the runtime versus solution quality tradeoff, so in the real-world, people can adapt these algorithms to whatever use case they need.

Finally, I want to support distance function complexities from $\mathcal{O}(1)$ to $O(|V| + |E|)$. An algorithm for a constant-time distance will run too slowly for a distance function that's linear in graph size, so I will design heuristics to support both.

In total, I will need to design and test three different heuristics:

- Sparse $|V| \leq 3 \times 10^4$
- Sparse $|V| > 3 \times 10^4$, d is $\mathcal{O}(1)$
- Sparse $|V| > 3 \times 10^4$, d is $\mathcal{O}(|V| + |E|)$

I will also need a brute force algorithm for small graphs. I will test each algorithm across varying graphs, θ, ϵ, and d.

Demsar et al. (2006) suggests Wilcoxon signed-rank test for comparing two methods and Friedman test for multiple.

Eren. et al., 2012 provides precedent for varying hyperparameters and reporting parameter sensitivity.

1.5 Future Work

This plan is missing a reduction plan, the actual pseudocode of the algorithms, and the hardware I'll be using for my embedding pipeline. I didn't have time this weekend to fully test redutions or algorithms (though I got started), and I am still unsure where I'll get the hardware required for embedding.

If I still have time after running these experiments, I would like to focus on:

- Creating an indexing structure to store large ϵ-CC solutions given different ϵ and θ
- I have an idea for an intersection parameter I that requires at least I right-side nodes that are a neighbor of every left-side node in an ϵ-CC. How does using this parameter change the graph structure? (I'll have to

decide soon if I want to include this, since otherwise there's really no point in running every single one of these experiments without $I > 0$).

- Can a GNN be trained to reduce the graph?
- Can a coarsening approach be used beyond normal reductions?